```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

Task 1

```
data = load_breast_cancer(as_frame=True)
df = data.frame

print("First Five Rows:")
print(df.head())

print("\nDataset Shape:", df.shape)
```

```
First Five Rows:
   mean radius  mean texture  mean perimeter  mean area  mean smoothness  \
0        17.99         10.38          122.80     1001.0          0.11840
1        20.57         17.77          132.90     1326.0          0.08474
2        19.69         21.25          130.00     1203.0          0.10960
3        11.42         20.38           77.58      386.1          0.14250
4        20.29         14.34          135.10     1297.0          0.10030

   mean compactness  mean concavity  mean concave points  mean symmetry  \
0           0.27760          0.3001              0.14710         0.2419
1           0.07864          0.0869              0.07017         0.1812
2           0.15990          0.1974              0.12790         0.2069
3           0.28390          0.2414              0.10520         0.2597
4           0.13280          0.1980              0.10430         0.1809

   mean fractal dimension  ...  worst texture  worst perimeter  worst area  \
```

```
0                 0.07871  ...          17.33           184.60      2019.0
1                 0.05667  ...          23.41           158.80      1956.0
2                 0.05999  ...          25.53           152.50      1709.0
3                 0.09744  ...          26.50            98.87       567.7
4                 0.05883  ...          16.67           152.20      1575.0

   worst smoothness  worst compactness  worst concavity  worst concave points  \
0            0.1622             0.6656           0.7119                0.2654
1            0.1238             0.1866           0.2416                0.1860
2            0.1444             0.4245           0.4504                0.2430
3            0.2098             0.8663           0.6869                0.2575
4            0.1374             0.2050           0.4000                0.1625

   worst symmetry  worst fractal dimension  target
0          0.4601                  0.11890       0
1          0.2750                  0.08902       0
2          0.3613                  0.08758       0
3          0.6638                  0.17300       0
4          0.2364                  0.07678       0

[5 rows x 31 columns]

Dataset Shape: (569, 31)
```

TASK 2: DATA PREPROCESSING

```
X = df.drop(columns=["target"])

print("\nMissing Values:")
print(X.isnull().sum().sum())

X = X.fillna(X.mean())

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("\nData preprocessing completed.")
```

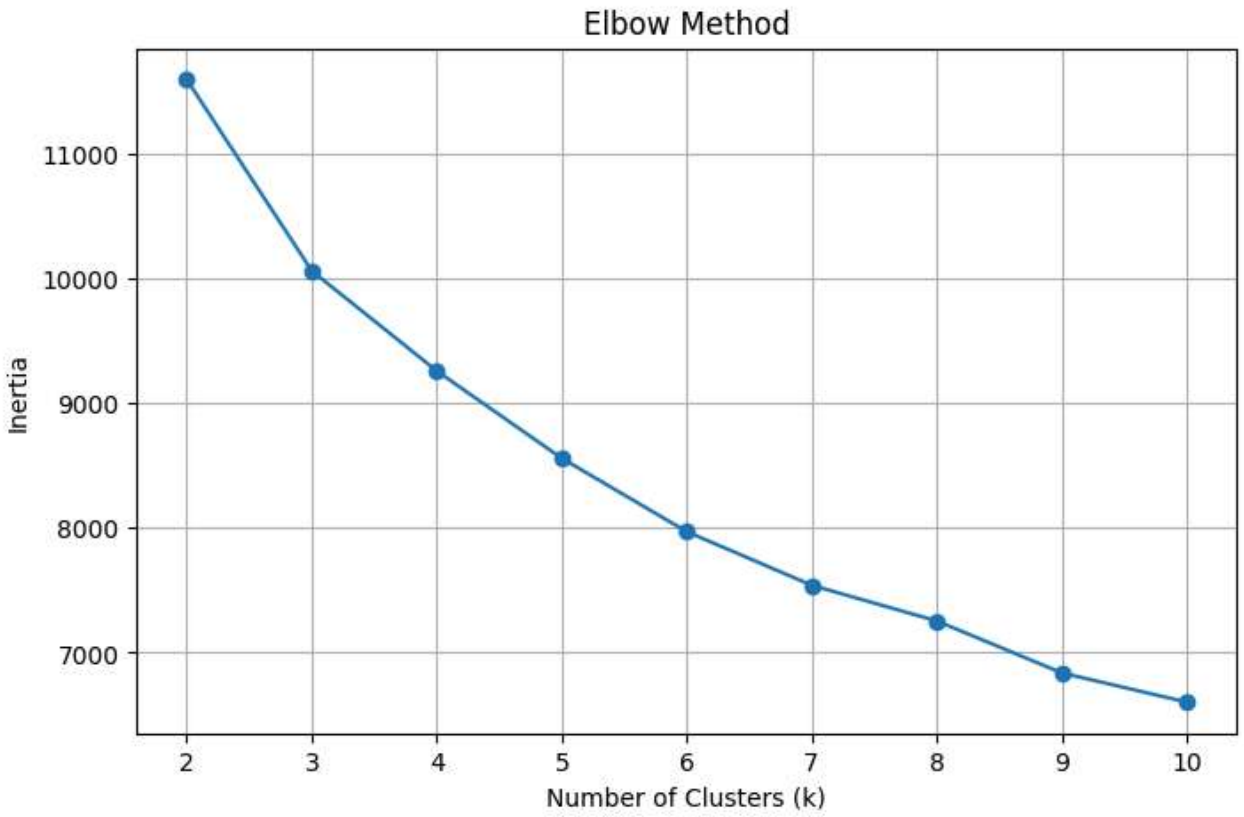
```

Missing Values:
0

Data preprocessing completed.
```

TASK 3: CLUSTER SELECTION

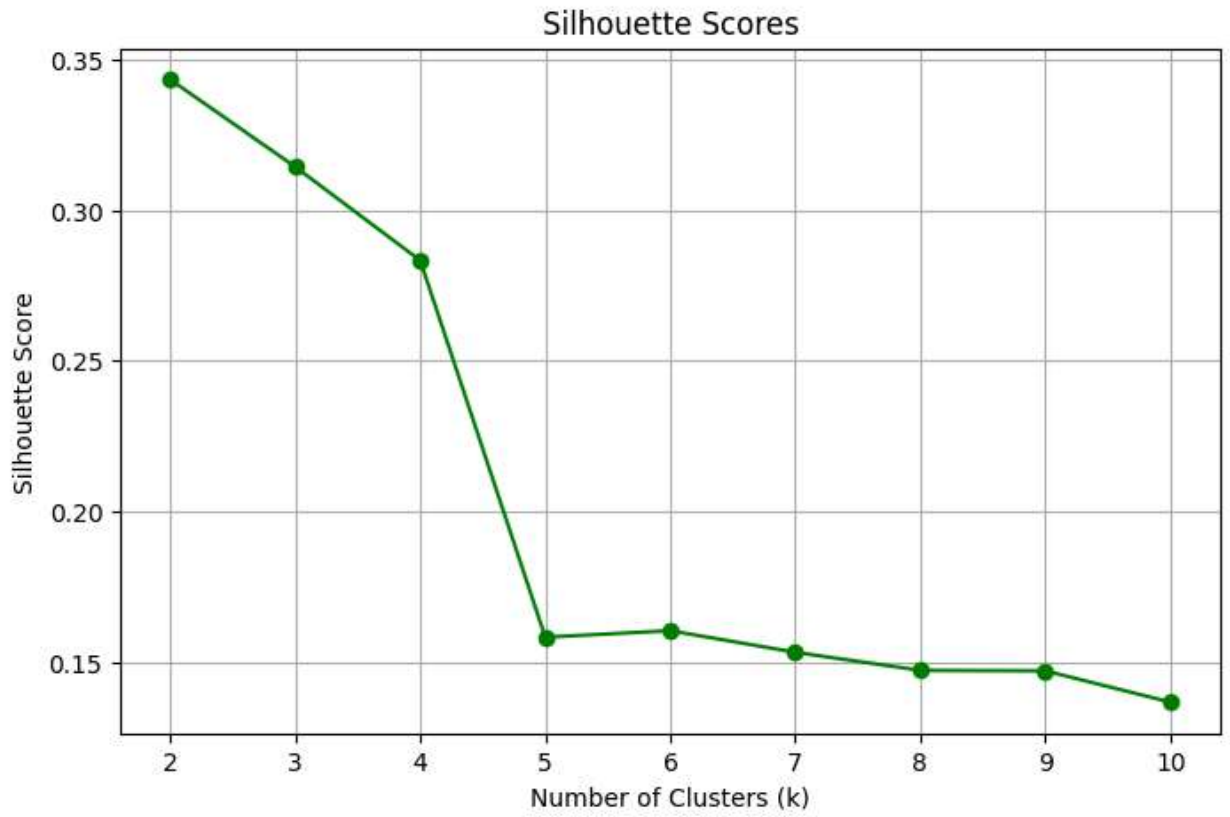
```
inertia = []
silhouette_scores = []

K = range(2, 11)

for k in K:
    model = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )

    labels = model.fit_predict(X_scaled)

    inertia.append(model.inertia_)
    silhouette_scores.append(
        silhouette_score(X_scaled, labels)
    )
```

Task 4: Visualization

```
plt.figure(figsize=(8,5))
plt.plot(K, inertia, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertia")
plt.grid(True)
plt.show()
```




```
plt.figure(figsize=(8,5))
plt.plot(K, silhouette_scores, marker='o', color='green')
plt.title("Silhouette Scores")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Silhouette Score")
plt.grid(True)
plt.show()
```




```
print("\nSilhouette Scores")

for k, score in zip(K, silhouette_scores):
    print(f"k = {k} --> {score:.4f}")
```

```
Silhouette Scores
k = 2 --> 0.3434
k = 3 --> 0.3144
k = 4 --> 0.2833
k = 5 --> 0.1582
k = 6 --> 0.1604
k = 7 --> 0.1532
k = 8 --> 0.1472
k = 9 --> 0.1470
k = 10 --> 0.1367
```

Task 5: Model

```
best_k = K[np.argmax(silhouette_scores)]

print("\nBest k based on Silhouette Score:", best_k)

kmeans = KMeans(
    n_clusters=best_k,
    random_state=42,
    n_init=10
)

clusters = kmeans.fit_predict(X_scaled)

df["Cluster"] = clusters
```

```
Best k based on Silhouette Score: 2
```

```
plt.figure(figsize=(8,6))

plt.scatter(
    X_scaled[:,0],
    X_scaled[:,1],
    c=clusters,
    s=40 ,
    cmap='cool'

)

plt.title(f"K-Means Clustering (k={best_k})")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.legend()
plt.show()
```

```
/tmp/ipykernel_818/3437984018.py:15: UserWarning: No artists with labels found to put in legend.  Note that artists whose label
  plt.legend()
```